HT_thermometer Example Application Note

Revision History

Date	Version	Description
March 29, 2021	0.50	Initial version created.
April 14, 2021	0.51	Updated format, no content change.

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This document is subject to change without notice.

Table of Contents

Overview 3
Quick Start 3
Sensor 4
BME680 4
ADT7420 7
Application State 9
Parameter Configuration 10
BLE Advertisement 10
BLE GAP Parameter 13
Device Information Service 13
Health Thermometer Service 14
Connection Parameter Update 14
GATT Service Client Characteristic Configuration 16

List of Figures

Figure 1 - nRF Toolbox Screenshots
Figure 2 - EVK I2C0 CLK and Data Pins
Figure 3 - CLK and Data Jumpers
Figure 4 - EVK I2C1 CLK and Data Pins
Figure 5 - Application States
Figure 6 - Application Discovered by Android APP
Figure 7 - GATT Service Discovery by Android APP
Figure 8 - Temperature Measurement
Figure 9 - BLE Advertisement Default Configuration
Figure 10 - Keil Configuration Wizard
Figure 11 - BLE GAP Parameter Default Configuration

CONFIDENTIAL

Overview

This example application implements a health thermometer profile (HTP), health thermometer service (HTS) and device information service (DIS) using BME680 or ADT7420 sensor.

Quick Start

platform/atmx/ATMxxxx/examples/HT_thermometer
$ make run_all

Device name is “Atm-HTPT”
Figure 1 shows screenshots when using nRF Toolbox Android APP.

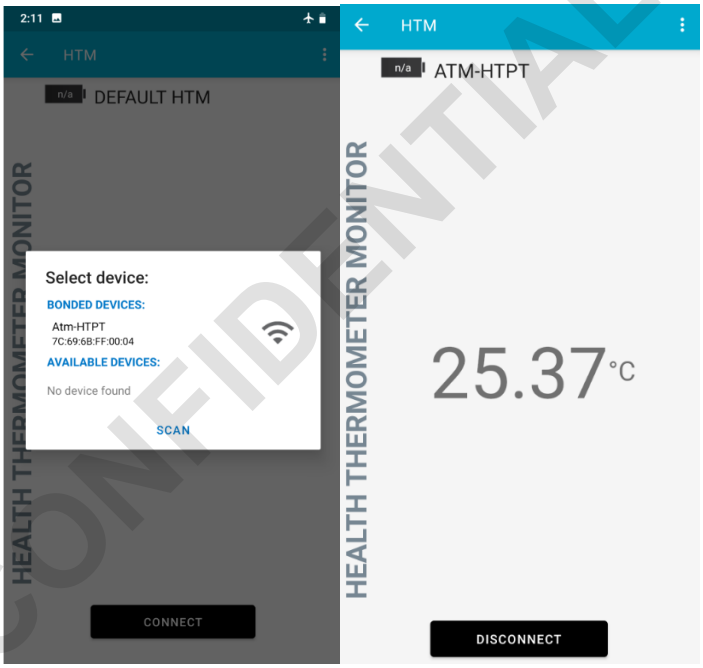


Figure 1 - nRF Toolbox Screenshots

Note:

1. The example application will update temperature every 2 secs (**APP_CFG_HT_MEAS_INTV_DFT**) by default. nRF APP can follow the HTP standard (measurement interval change GATT interface) to make modifications.

2. If you modify source code and want to program the flash, You can use “make run” instead of “make run_all”. “make run_all” will also erase Flash NVDS data including bonding information. If you erase the bonding information, you will also need to remove the bonding in the APP site.

3. Use “make DEBUG:=” to build release version

CONFIDENTIAL

Sensor

The example can support two sensors: BMR680 or ADT7420. The default sensor is BME680. You can select the sensor by makefile:

- ADT7420: enable CFG_ADT7420_SENSOR
- BME680: enable CFG_BME680_SENSOR

BME680

The default driver of BME680 in SDK is using I2C0 with P9_I2C0_SCK and P10_I2C0_SDA as clk and data pin. See Figure 2 and Figure 3.

ATM2/ATM3 5x5 EVK:

	SCL	SDA	VDD	GND
50 pin header GPIO	39 P9	40 P10	2 N/A	1 N/A

Figure 2 - EVK I2C0 CLK and Data Pins



Figure 3 - CLK and Data Jumpers

CONFIDENTIAL

Function APIs in bme680.c:

```
/**
 ******************************************************************************
 * @brief Initialize i2c and sensor.
 ******************************************************************************
 */
void bme680_init(void);

/**
 ******************************************************************************
 * @brief Read temperature from sensor
 *
 * @return sensor temperature in Celsius.
 ******************************************************************************
 */
float bme680_get_temp(void);

/**
```

CONFIDENTIAL

I2C related APIs in bme680.c:

```
/**
 *****************************************************************
 * @brief read data from sensor
 *
 * @param[in] reg_addr start address to read data
 * @param[out] data pointer to a block of memeory
 * @param[in] length size to be read
 * @return non-negative: # of bytes read
 * negative: failure
 *****************************************************************
 */
__attribute__((nonnull(2)))
int bme680_read(uint8_t reg_addr, uint8_t *data, int length);

/**
 *****************************************************************
 * @brief read one byte from sensor
 *
 * @param[in] reg_addr start address to read data
 * @param[out] data pointer to data address
 * @return true if successful
 *****************************************************************
 */
__attribute__((nonnull(2)))
bool bme680_read_byte(uint8_t reg_addr, uint8_t *data);

/**
 *****************************************************************
 * @brief read one word from sensor
 *
 * @param[in] reg_addr start address to read data
 * @param[out] data pointer to data address
 * @return true if successful
 *****************************************************************
 */
__attribute__((nonnull(2)))
bool bme680_read_word(uint8_t reg_addr, uint16_t *data);

/**
 *****************************************************************
 * @brief write data to sensor
 *
 * @param[in] reg_addr start address to write data
 * @param[in] data pointer to a block of memeory
 * @param[in] length size to be written
 * @return non-negative: # of bytes read
 * negative: failure
 *****************************************************************
 */
__attribute__((nonnull(2)))
int bme680_write(uint8_t reg_addr, uint8_t const *data, int length);

/**
 *****************************************************************
 * @brief write one byte to sensor
 *
 * @param[in] reg_addr start address to write data
 * @param[in] data data to be written
 * @return true if successful
 *****************************************************************
 */
bool bme680_write_byte(uint8_t reg_addr, uint8_t data);

/**
 *****************************************************************
 * @brief write one word to sensor
 *
 * @param[in] reg_addr start address to write data
 * @param[in] data data to be written
 * @return true if successful
 *****************************************************************
 */
bool bme680_write_word(uint8_t reg_addr, uint16_t data);
```

ADT7420

The default driver of ADT7420 in SDK is using I2C1 with P12_I2C1_SCK and P13_I2C1_SDA as clk and data pin. See Figure 4.

ATM2/ATM3 5x5 EVK:

	SCL	SDA	VDD	GND
50 pin header GPI0	42 P12	3 P13	2 N/A	1 N/A

Figure 4 - EVK I2C1 CLK and Data Pins

Function APIs in adt7420.c:

```
/**
 * @brief Start temperature measurement from sensor
 */
void adt7420_start_temp(void);

/**
 * @brief Read temperature from sensor
 * @return sensor temperature in Celsius.
 */
float adt7420_get_temp(void);
```

I2C related APIs in adt7420.c:

```
/**
 * @brief read data from sensor
 * @param[in] reg_addr start address to read data
 * @param[out] data    pointer to a block of memeory
 * @param[in] length   size to be read
 * @return non-negative: # of bytes read
 * negative: failure
 */
int adt7420_read(uint8_t reg_addr, uint8_t *data, int length);

/**
 * @brief read one byte from sensor
 * @param[in] reg_addr  start address to read data
 * @param[out] data     pointer to data address
 * @return Success: true
 * Failure: false
 */
bool adt7420_read_byte(uint8_t reg_addr, uint8_t *data);

/**
 * @brief read one word from sensor
 * @param[in] reg_addr  start address to read data
 * @param[out] data     pointer to data address
 * @return Success: true
 * Failure: false
 */
bool adt7420_read_word(uint8_t reg_addr, uint16_t *data);
```

```
/**
 * @brief read one word from sensor
 * @param[in] reg_addr  start address to read data
 * @param[out] data     pointer to data address
 * @return Success: true
 * Failure: false
 */
bool adt7420_read_word(uint8_t reg_addr, uint16_t *data);

/**
 * @brief write data to sensor
 * @param[in] reg_addr  start address to write data
 * @param[in] data      pointer to a block of memeory
 * @param[in] length    size to be written
 * @return non-negative: # of bytes read
 * negative: failure
 */
int adt7420_write(uint8_t reg_addr, uint8_t *data, int length);

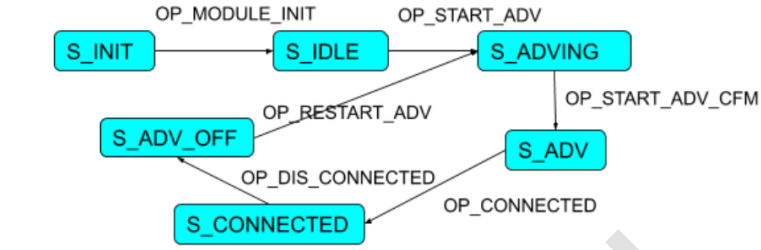
/**
 * @brief write one byte to sensor
 * @param[in] reg_addr  start address to write data
 * @param[in] data      data to be written
 * @return Success: true
 * Failure: false
 */
bool adt7420_write_byte(uint8_t reg_addr, uint8_t data);

/**
 * @brief write one word to sensor
 * @param[in] reg_addr  start address to write data
 * @param[in] data      data to be written
 * @return Success: true
 * Failure: false
 */
bool adt7420_write_word(uint8_t reg_addr, uint16_t data);
```

CONFIDENTIAL

Application State

After boot up, the device will start to send advertisements and wait for a connection. After connection, the device will stop advertising and wait for the GATT service operation. After the disconnection, the device will start to send advertisements. See Figure 5.




```
state_entry const s_tbl[] = {
    {S_OP(S_INIT, OP_MODULE_INIT), S_IDLE, meter_mmi_init},
    {S_OP(S_IDLE, OP_START_ADV), S_ADVING, meter_mmi_start_adv},
    {S_OP(S_ADVING, OP_START_ADV_CFM), S_ADV, meter_mmi_start_on},
    {S_OP(S_ADV, OP_CONNECTED), S_CONNECTED, meter_mmi_connected},
    {S_OP(S_CONNECTED, OP_DIS_CONNECTED), S_ADV_OFF, meter_mmi_disconnected},
    {S_OP(S_ADV_OFF, OP_RESTART_ADV), S_ADVING, meter_mmi_adv_restart_adv}
};
```

Figure 5 - Application States

Figure 6 is screenshot using “nRF connect” Android APP to discover this Application.

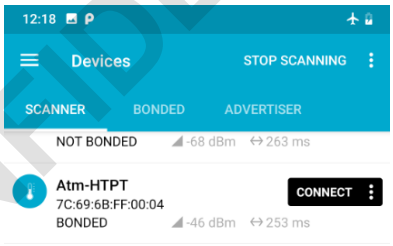
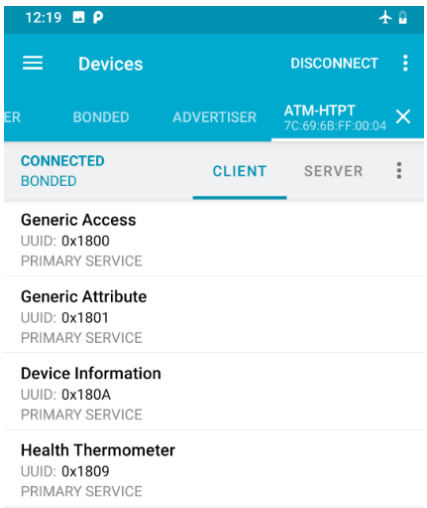


Figure 6 - Application Discovered by Android APP

Figure 7 shows screenshot using “nRF connect” Android APP for GATT service discovery.



CONFIDENTIAL

Figure 7 - GATT Service Discovery by Android APP

Enable indication for temperature measurement of health thermometer service. See Figure 8.



Figure 8 - Temperature Measurement

Parameter Configuration

BLE Advertisement

The default configuration for BLE advertisement is loaded from “**param_gap_adv.h**”. There are three option flags to control whether datas are loaded from “param_gap_adv.h” or Flash NVDS Tag Data. Advertisements interval is 250 ms , TX power is 0 dBm and there is no advertisement timeout. See Figure 9.

CONFIDENTIAL

```
static void meter_mmi_init(void)
{
    ATM_LOG(V, "%s: ", __func__);

    // Enable Device Information Service
    atm_gap_prf_reg(BLE_DISS_MODULE_NAME, app_dis_param());

    // Enable Battery Service
    atm_gap_prf_reg(BLE_BASS_MODULE_NAME, app_bass_param());

    // Enable Health Thermometer Service
    atm_gap_prf_reg(BLE_HTPT_MODULE_NAME, app_htpt_param());

    // GAP initialize
    atm_gap_start(atm_gap_param_get(), &gap_cbfn);

    sw_timer_set(tid_update_time, APP_UPDATE_TIME_INV_CS);
    app_htpt_timestamp_init(&mmi_time_stamp);


#ifndef ENABLE_USER_ADV_PARAM_SETTING
    adv_create = atm_adv_create_param_nvds();
#else
    adv_create = atm_adv_create_param_get(0);
#endif  // ENABLE_USER_ADV_PARAM_SETTING

#ifndef ENABLE_USER_ADV_DATA_SCANRSP
    adv_data = atm_adv_advdata_param_nvds();
    scan_data = atm_adv_scandata_param_nvds();
#else
    adv_data = atm_adv_advdata_param_get(0);
    scan_data = atm_adv_scandata_param_get(0);
#endif // ENABLE_USER_ADV_DATA_SCANRSP

#ifndef ENABLE_USER_ADV_TIMEOUT
    adv_start = atm_adv_start_param_nvds();
#else
    adv_start = atm_adv_start_param_get(0);
#endif // ENABLE_USER_ADV_TIMEOUT

    // get txpwr_dbm from create adv. parameter
    txpwr_dbm = adv_create->adv_param.max_tx_pwr;
    ATM_LOG(V, "  txpwr_dbm = %d dbm", txpwr_dbm);
}
```

Figure 9 - BLE Advertisement Default Configuration

“Param_gap_adv.h” is compatible with the Keil Configuration Wizard. It will be easier to use to configure the settings. See Figure 10.

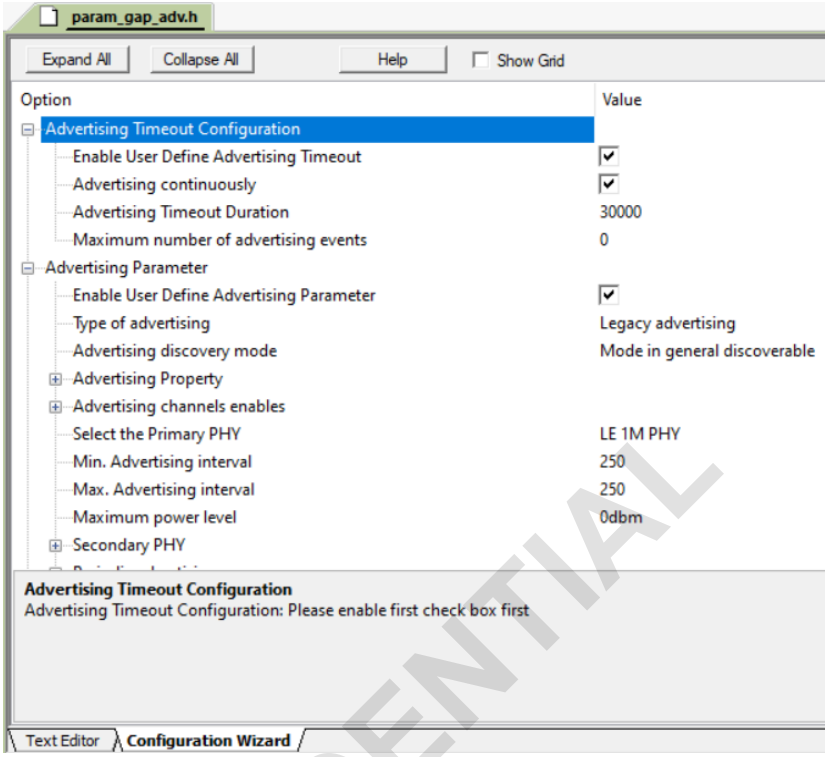


Figure 10 - Keil Configuration Wizard

Use CFG_ADV0_DATA_ADV_PAYLOAD and CFG_ADV0_DATA_SCANRSP_PAYLOAD to set advertisement payload and scan response payload.

```
#define CFG_ADV0_DATA_ADV_PAYLOAD /*Service
UUID*/0x03,0x02,0x09,0x18,/*Appearance*/0x03,0x19,0x00,0x03

#define CFG_ADV0_DATA_SCANRSP_PAYLOAD /*Complete Local
Name*/0x09,0x09,0x41,0x74,0x6D,0x2D,0x48,0x54,0x50,0x54
```

Note:

1. Complete local name and appearance are configured, see code above.
2. Flags (for example, 0x02 0x01 0x06) elements will be added automatically.

CONFIDENTIAL

BLE GAP Parameter

The default configuration for BLE GAP parameter is loaded from "**param_gap.h**", see Figure 11.

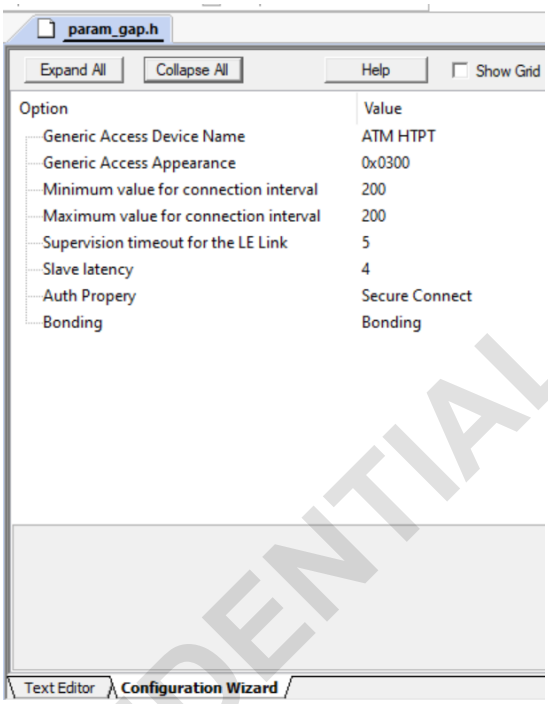


Figure 11 - BLE GAP Parameter Default Configuration

Below are the default setting:

```
Device Name of Generic Access profile - "ATM HTPT"
Appearance of Generic Access profile - Generic Thermometer (0x0300)
Connection interval - 250ms (200*1.25ms)
Supervision timeout - 5sec (500*10ms)
Slave latency - 4 connection event
Auth. property - Secure Connection
Bonding - Enable
```

Device Information Service

The default configuration for device information service is loaded from "**app_config.h**"

```
#define APP_DIS_MANUFACTURER_NAME ("Atmosic Tech.")
#define APP_DIS_MODEL_NB_STR ("HTP Thermometer")
#define APP_DIS_SERIAL_NB_STR ("1.0.0.0")
#define APP_DIS_FIRM_REV_STR ("1.0.0.0")
#define APP_DIS_HARD_REV_STR ("1.0.0")
#define APP_DIS_SW_REV_STR ("1.0.0.0")
```

CONFIDENTIAL

Health Thermometer Service

The default configuration for health thermometer service is loaded from "**app_htpt.h**"

```
/// HTPT Feature Flags
//<h> APP_CFG_HTPT_FEATURES
// <i> HTPT Feature Flags
// <q.0> Temperature Type Char. is supported
// <q.1> Intermediate Temperature Char. is supported
// <q.2> Measurement Interval Char. is supported
// <q.3> Measurement Interval Char. supports indications
// <q.4> Measurement Interval Char. is writable
#define APP_CFG_HTPT_FEATURES 0x1F
//</h>

/// Measurement Interval Value Min
#define APP_CFG_HT_MEAS_INTV_MIN         1
/// Measurement Interval Value Max
#define APP_CFG_HT_MEAS_INTV_MAX         300
/// Measurement Interval Value Default
#define APP_CFG_HT_MEAS_INTV_DFT         2
/// Temperature Type
#define APP_CFG_HT_TEMP_TYPE             HTP_TYPE_BODY
/// Intermediate Temperature update time
#define APP_CFG_HT_INTM_TEMP_INTV_CS     100
```

CONFIDENTIAL

Connection Parameter Update

In the app_gap.c, **param_nego** structure is used to operate the connection parameter update procedure. The example application will perform the connection parameter update procedure after connection is established. See the following:

```
void app_gap_conn_ind(uint8_t conidx, atm_connect_info* param)
{
    ATM_LOG(V, "%s", __func__);
    ATM_LOG(N, " - current peer addr 0x%02X:0x%02X:0x%02X:0x%02X:0x%02X:0x%02X",
       param->peer_addr.addr[5], param->peer_addr.addr[4],
       param->peer_addr.addr[3], param->peer_addr.addr[2],
       param->peer_addr.addr[1], param->peer_addr.addr[0]);

    atm_debug_print_conn_param(param);
    atm_gap_connect_accept(conidx);
    atm_gap_connect_param_nego(conidx, (atm_gap_param_nego_t *) &param_nego);
}
```

Framework will have the callback function (**param_nego_cfm**) when this procedure is complete. The example application can use **app_gap_connect_param_nego** or **atm_gap_connect_param_nego** to perform the connection parameter update procedure when you want to negotiate the parameter.

Note:

1. Only allow a single instance to perform connection parameter procedure. It means when you use the API to trigger the procedure, it needs to make sure the previous procedure's callback returns.
2. Currently, the **param_nego** is using CONST type, if you want to change the parameter setting in different scenarios, please remember to remove CONST type.

CONFIDENTIAL

GATT Service Client Characteristic Configuration

The **app_htpt_ind_ntf** is the callback function to handle CCC (Client Characteristic Configuration). When enabling CCC, the example application will create a software timer to read sensor data and respond to GATT client roles.

```
static void app_htpt_ind_ntf(uint8_t conidx,
    struct htpt_cfg_indntf_ind const *param)
{
    uint8_t link_idx = app_htpt_get_linkidx_by_conidx(conidx);
    if (link_idx >= BLE_CONNECTION_MAX) {
       ATM_LOG(E, "%s - %u", __func__, conidx);
       return;
    }
    ATM_LOG(N, "%s- conidx=%u ntf_ind_cfg=%#x, %#x", __func__, conidx,
       param->ntf_ind_cfg, app_htpt_sts[link_idx].ind_ntf_value);
    ASSERT_ERR(app_htpt_sts[link_idx].valid);

#if PLF_DEBUG
    if (param->ntf_ind_cfg & APP_HTPT_STABLE_MEAS_IND) {
       ATM_LOG(D, "  - APP_HTPT_STABLE_MEAS_IND");
    }
    if (param->ntf_ind_cfg & APP_HTPT_INTERM_MEAS_NTF) {
       ATM_LOG(D, "  - APP_HTPT_INTERM_MEAS_NTF");
    }
    if (param->ntf_ind_cfg & APP_HTPT_MEAS_INTV_IND) {
       ATM_LOG(D, "  - APP_HTPT_MEAS_INTV_IND");
    }
#endif
    uint8_t tmp_cfg = app_htpt_sts[link_idx].ind_ntf_value ^ param->ntf_ind_cfg;
    app_htpt_sts[link_idx].ind_ntf_value = param->ntf_ind_cfg;

    if (tmp_cfg & APP_TIMER_MEAS_TEMP_IND) {
       app_htpt_timer_set(link_idx, APP_HTPT_TIMER_MEAS_TEMP);
    }
    if (tmp_cfg & HTPT_CFG_INTERM_MEAS_NTF) {
       app_htpt_timer_set(link_idx, APP_HTPT_TIMER_INTERM_MEAS);
    }
}
```

The example application will register a related callback function with app_htpt_reg_sensor_temp to get temperature through the callback function.

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com